

AGENTOS ARCHITECTURE SPECIFICATION

v3.0.1 — Logic-corrected build-ready edition

AgentOS v3.0

Supervisor + Cognitive Engine: a build-ready architecture for autonomous agent workloads on a single node.

Scope: single machine (Windows 11 + WSL2 / Linux) | **Delivery:** phased MVP | **Replaces:** v3.0.0

Correction principle: this revision preserves the original architecture and terminology, but removes guarantees or control flows that are logically stronger than the mechanisms described.

0. Logic Corrections Applied

Issue	Corrected logic
External side effects	A post-tool reflection cannot prevent a state-changing side effect. State-changing actions now require preflight policy/approval checks and postcondition verification.
Task-loss guarantee	"No task lost" is conditional on durable checkpoints. External actions can still duplicate after a crash unless they use idempotency keys or reconciliation.
Deterministic replay	A log alone is insufficient. Replay records model/tool outputs, randomness and relevant environment state; external side effects are mocked or reconciled rather than blindly repeated.
Hot-patching	Loaded code is not mutated in place. New versions are tested and activated at task boundaries; in-flight executions keep their existing version.
Platform enforcement	seccomp and cgroups v2 are Linux mechanisms. Windows/WSL2 requires an explicit host/VM boundary and Windows-side process controls; the supervisor must not imply identical enforcement on every platform.
Secrets	Proxy injection prevents secrets from entering model context only if the proxy also prevents them from being returned in tool output/logs and scopes each credential to its allowed destination.
Write permissions	"Read-only by default" is clarified: agents may write only inside explicitly authorized workspaces; host writes, destructive actions and external side effects remain separately gated.
Auto-quarantine	Regression automation quarantines a failing tool version, with dependency checks and fallback; it does not silently disable a safety-critical control.

1. Core: Safety Supervisor + Cognitive Engine

- **Safety Supervisor** (privileged userspace process, not a kernel): enforces available platform controls, including Linux seccomp profiles and cgroups v2 quotas (RAM, CPU, PIDs), plus watchdogs and an emergency stop. On Windows/WSL2, platform-specific host/VM controls must provide the corresponding boundary; seccomp/cgroups claims apply only where supported. Supervisor code and policy are read-only to agents.
- **Planner:** compiles goals into a task DAG with prerequisite gates, durable checkpoints and validation checkpoints.
- **Action Gate:** before every state-changing or external action, evaluates capability, policy, scope, approval requirement, spend limit, egress destination and idempotency/retry policy.
- **Reflection / Postcondition Gate:** validates tool output and the observed postcondition before dependent state is committed or a user-facing claim is made. For read-only tools, output validation is sufficient; for mutating tools, it follows preflight authorization.
- **Context Governor:** pins key entities, summarizes older turns, archives full text and retrieves it on demand.

2. Cost and Token Governor

- **Model routing:** cheap/local model for parsing, summaries and tool-argument extraction; stronger model for planning and high-risk steps.
- **Prompt caching:** stable prefix contains system prompt, tool schemas and persona; volatile content is last.
- **Spend limits:** per-task and per-day token/dollar cap; auto-downgrade where policy permits, then halt at the limit.
- **Output discipline:** JSON-schema outputs and short formats. Cache only results whose cache key and semantics make reuse safe; do not assume LLM responses are deterministic.
- **Lazy tool loading:** load only schemas needed for the current step.

3. Action Space

Capability	Implementation
Sandbox tiers	Rootless Docker (default); gVisor for workloads requiring stronger isolation where supported; Firecracker optional where KVM is available.
Shell	PTY manager with streamed output, job supervision, exit-code capture and explicit workspace/write scope.

Capability	Implementation
Browser	Playwright/CDP first; vision fallback (screenshot + element marks) for GUIs. Browser actions that send, purchase or otherwise mutate external state pass the Action Gate.
Code edits	Diff-based patches in a designated worktree; tests before commit; rollback on failure. Repository publication/merge remains separately authorized.

4. Memory

- **Working:** context window plus pinned entities.
- **Long-term:** SQLite (preferences, task state, run history) with a vector extension for episodic recall.
- **Procedural:** tool recipes and failure notes as versioned files.
- **Later:** knowledge graph once retrieval quality demands it.
- **Durability rule:** task state is checkpointed before a retryable external action when practical; external actions use idempotency keys or reconciliation where the provider supports them.

5. Policy and Security

Control	Behaviour
Capability tiers	Read-only by default. Agents may write only inside explicitly authorized workspaces. Human approval is required for host writes, destructive deletes, payments, external sends and other policy-defined consequential actions.
Policy as code	Single <code>agentos.yaml</code> defines routes, quotas, egress allowlist, per-tool permissions, write scopes, approval rules and retry/idempotency policy.
Egress proxy	Domain allowlist; short-lived, destination-scoped secrets injected at the proxy. Proxy strips/redacts credentials from outputs and logs so secrets do not re-enter model context.
Audit log	Append-only, hash-chained record of tool calls, approvals, policy decisions, model/tool versions and spend events.
Injection defense	All tool and web output is untrusted data, never instructions. PII/secrets filtering runs before model-visible output and before user-facing output.
Kill / containment	Emergency stop terminates the agent's controlled processes and revokes capabilities. It is not described as a kernel primitive and does not substitute for a sandbox boundary.

6. Extensibility

- **MCP:** runtime server registration; each server runs under the same capability, egress and approval tiers.
- **Personas:** ReAct, Plan-and-Solve, Reflexion via flag. Prompt overlays provide domain/compliance packs.
- **Middleware:** pre-action and post-action hooks. Observability via OpenTelemetry to Langfuse, LangSmith or Grafana.

7. Self-Evolution (guarded)

- **Skill crystallization:** repeated action patterns become scripts only after tests, sandbox execution and human approval.
- **Nightly regression:** failing tool *versions* are quarantined after defined failure criteria, with dependency checks and a known-good fallback where available.
- **Self-patching:** agent opens a branch and PR with tests; a human merges; never auto-deploys; safety and policy enforcement code is excluded from agent modification.
- **Consensus:** multi-model voting may add evidence for high-consequence actions, under a cost cap, but never overrides policy, approval or capability controls.

8. Upgrades and Reliability

Operation	Target	Mechanism
Supervisor/worker restart	Seconds; recover checkpointed work	Drain where possible; persist task state before risky boundaries. For external actions, use idempotency keys/reconciliation to prevent duplicate effects.
Model swap	Next eligible turn	Provider-neutral message format via LiteLLM; fallback chain; preserve required structured state.
Tool hot-patch	No restart for future invocations	Versioned load; AST/static checks and tests; activate only at task boundaries. In-flight executions retain their current version.
Vector re-embed	No query downtime	Shadow index, validate, then atomic index switch.
DB migration	No data loss	Versioned migrations; validated snapshot/backup before apply; rollback or restore procedure.

Operation	Target	Mechanism
Self-patch	Never auto-prod	Canary instance, tests, human merge; deployment remains outside the agent's authority.

9. Evals and Safety Net

- **Golden task suite:** 20–30 representative tasks run on every change; track success rate, cost, steps and policy violations.
- **Trace replay:** failed runs can be replayed from recorded model/tool outputs, random seeds where applicable, configuration and environment metadata. External side effects are stubbed or reconciled rather than blindly re-executed.
- **Red-team set:** prompt-injection and sandbox-escape attempts remain permanent tests.

10. Build Order

Phase	Deliverables
1. MVP	Supervisor, one sandbox tier, LiteLLM routing, SQLite memory, audit log, cost caps, durable task checkpoints.
2. Safety	Egress proxy, approval tiers, write scopes, rollback, watchdog, idempotency/reconciliation hooks.
3. Capability	PTY, browser, MCP, diff-based code edits.
4. Quality	Eval suite, trace capture/replay, personas.
5. Evolution	Skill crystallization, then guarded self-patching.
6. Later	gVisor/Firecracker tiers as platform permits, graph memory, multi-node.

Result: v3.0.1 removes the principal circularity and overclaiming in v3.0.0 while retaining its single-node, phased-MVP design. The most important invariant is: *authorize before side effect; verify after side effect; checkpoint before retry; never equate a log with deterministic replay.*